Webtechnologien – Praktikum 03

CSS-Grundlagen

Übungsaufgabe

Lernziele

- Ein externes Stylesheet einbinden und mit Custom Properties arbeiten
- Selektoren und Klassen gezielt einsetzen und die Kaskade nachvollziehen
- Abstände und Rahmen mit dem Box-Modell gestalten
- Layouts mit Flexbox und Grid umsetzen
- Eine Website nach dem Prinzip Mobile first responsiv machen

Die Aufgabe ist eine unbewertete **Übungsaufgabe**.

Hinweise zur Abgabe

Es erfolgt keine Abgabe.

Nutzung von KI-Werkzeugen

Bitte lösen Sie die Aufgaben ohne KI-Werkzeuge wie ChatGPT, Claude oder Copilot. Die Übung ist unbewertet – ihr einziger Zweck ist, dass Sie die Grundlagen selbst verstehen und anwenden. Wenn eine KI den Code schreibt, lernen Sie dabei nichts, und dieses Fundament fehlt Ihnen in den späteren Blöcken.

Deaktivieren Sie dafür auch automatische Code-Vorschläge im Editor, zum Beispiel GitHub Copilot in VS Code. Wenn Sie nicht weiterkommen, fragen Sie Ihre Nachbarn, die Betreuung im Praktikum oder schlagen Sie in den MDN Web Docs nach.

Vorbereitung

Sie gestalten Ihre Website aus Praktikum 2. Sie benötigen:

- Ihr Repository `praktikum-02` mit Ihrer eigenen Website (Teil 3).
- Falls Teil 3 aus Praktikum 2 noch nicht fertig ist: Holen Sie das zuerst nach – ohne HTML gibt es nichts zu gestalten.

Teil 1: Stylesheet einbinden

Aufgabe 1.1: Projekt fortführen

a. Setzen Sie ein Tag `html_final` auf den finalen Stand ihrer HTML Implementierung aus der letzten Woche.

b. Die Speisekarte brauchen Sie nicht mehr. Löschen Sie `public/menu.html` mit `rm` und entfernen Sie den Link darauf aus der Navigation aller Seiten.

Aufgabe 1.2: Stylesheet anlegen

a. Legen Sie die Datei `public/css/style.css` an.

b. Binden Sie das Stylesheet im `head` aller Seiten mit `<link rel="stylesheet" href="…">` ein. Achten Sie auf den richtigen relativen Pfad: Von `index.html` aus lautet er `css/style.css`, von den Seiten im Ordner `projects` aus `../css/style.css`.

c. Prüfen Sie im Network-Tab, dass `style.css` auf jeder Seite mit Status 200 und dem Typ `text/css` geladen wird.

d. Bauen Sie absichtlich einen falschen Pfad ein. Was zeigt der Network-Tab, und wie sieht die Seite aus? Korrigieren Sie den Pfad anschließend wieder.

Aufgabe 1.3: Typografie und Farben

In diesem Schritt verwenden Sie nur Element-Selektoren, noch keine Klassen.

a. Definieren Sie in `:root` mindestens drei Farben und eine Schriftart als Custom Properties, zum Beispiel `--color-primary`, `--color-text`, `--color-background` und `--font-body`.

b. Gestalten Sie `body` mit `font-family`, `line-height`, `color` und `background-color` – jeweils mit `var()`.

c. Geben Sie Links eine Farbe und einen `:hover`-Zustand.

d. Ändern Sie eine Farbe in `:root` und beobachten Sie, wie sich alle Stellen gleichzeitig ändern.

e. Sichern Sie den Stand:

```
git add .
git commit -m "Add stylesheet with typography and colors"
```

Teil 2: Klassen, Box-Modell und Kaskade

Aufgabe 2.1: Klassen vergeben

a. Geben Sie dem Seitenrahmen auf allen Seiten Klassen: `site-header`, `site-nav` und `site-footer`.

b. Ergänzen Sie auf `index.html` eine Übersicht Ihrer Projekte: mindestens zwei `article`-Elemente mit Titel, kurzer Beschreibung und Link zur Detailseite. Geben Sie jedem `article` die Klasse `project-card`. Inhalte dürfen erfunden sein.

c. Wählen Sie Klassennamen, die beschreiben, was etwas ist – nicht, wie es aussieht.

Aufgabe 2.2: Box-Modell

a. Setzen Sie am Anfang des Stylesheets `box-sizing: border-box` für alle Elemente und entfernen Sie den Standard-Außenabstand von `body`:

```
*,
*::before,
*::after {
  box-sizing: border-box;
}

body {
```

```
  margin: 0;
}
```

b. Begrenzen Sie die Breite von `main` mit `max-width` und zentrieren Sie es mit `margin: 0 auto`.

c. Gestalten Sie `project-card` als Karte: Hintergrundfarbe, `padding`, `border`, `border-radius` und `margin-bottom`.

d. Untersuchen Sie eine Karte in den DevTools. Ändern Sie in der Box-Modell-Ansicht das `padding` direkt im Browser und übertragen Sie den Wert, der Ihnen gefällt, in Ihr Stylesheet.

Aufgabe 2.3: Die Kaskade erkunden

a. Sie haben in Aufgabe 1.3 allen Links eine Farbe gegeben. Ergänzen Sie jetzt eine Regel, die nur Links in der Navigation eine andere Farbe gibt:

```
.site-nav a {
  color: var(--color-text);
}
```

b. Untersuchen Sie einen Navigationslink im Styles-Panel. Welche Regel ist durchgestrichen, und warum?

c. Verschieben Sie die Regel `.site-nav a` im Stylesheet vor die Regel für `a`. Ändert sich etwas? Begründen Sie Ihre Beobachtung mit der Spezifität.

d. Sichern Sie den Stand:

```
git add .
git commit -m "Add classes and card styles"
```

Teil 3: Layout mit Flexbox und Grid

Aufgabe 3.1: Header und Navigation mit Flexbox

a. Machen Sie `site-header` zum Flex-Container: Seitenname links, Navigation rechts, vertikal zentriert.

b. Ordnen Sie die Liste in der Navigation mit Flexbox nebeneinander an, mit `gap` als Abstand und ohne Aufzählungspunkte.

Aufgabe 3.2: Karten im Raster mit Grid

a. Fassen Sie die Projektkarten auf `index.html` in einem Container mit der Klasse `project-list` zusammen und ordnen Sie sie mit Grid in zwei Spalten an.

b. Stellen Sie auf Ihrer Projekt-Detailseite die Definitionsliste (`dl`) mit Grid als zweispaltige Tabelle dar: Begriffe links, Beschreibungen rechts. Tipp: `grid-template-columns: max-content 1fr`.

c. Sichern Sie den Stand:

```
git add .
git commit -m "Add layout with Flexbox and Grid"
```

Teil 4: Responsive Design

Aufgabe 4.1: Probleme finden

a. Öffnen Sie in den DevTools die Gerätesymbolleiste (Strg+Shift+M bzw. Cmd+Shift+M) und stellen Sie eine Breite von 390 Pixeln ein.

b. Gehen Sie alle Seiten durch und notieren Sie, was auf dem Smartphone nicht funktioniert. Achten Sie besonders auf die Spalten und auf Bilder.

Aufgabe 4.2: Mobile first

a. Stellen Sie die Projektliste so um, dass sie standardmäßig nur eine Spalte hat. Ergänzen Sie eine Media Query, die ab einer Breite von `40rem` auf zwei Spalten umschaltet:

```
@media (min-width: 40rem) {
  .project-list {
    grid-template-columns: repeat(2, 1fr);
  }
}
```

b. Sorgen Sie dafür, dass Bilder nie breiter als ihr Container werden: `max-width: 100%` und `height: auto`.

c. Ordnen Sie im Header Seitenname und Navigation auf schmalen Bildschirmen untereinander an und erst ab `40rem` nebeneinander.

Aufgabe 4.3: Prüfen und veröffentlichen

a. Prüfen Sie alle Seiten bei 390, 768 und 1280 Pixeln Breite.

b. Sichern Sie den Stand und laden Sie alle Commits hoch:

```
git add .
git commit -m "Make layout responsive"
git push
```

Checkliste zum Abschluss

- Das Stylesheet wird auf allen Seiten geladen, auch in den Unterordnern.
- Farben und Schrift stehen als Custom Properties in `:root`.
- Ich kann im Styles-Panel erklären, warum eine Regel durchgestrichen ist.
- Ich kann `padding`, `border` und `margin` im Box-Modell unterscheiden.
- Header und Navigation nutzen Flexbox, die Projektkarten Grid.
- Die Seite funktioniert bei 390, 768 und 1280 Pixeln Breite.

CSS-Übersicht

Schreibweise	Bedeutung
`h1`	Element-Selektor: alle `h1`-Elemente
`.project-card`	Klassen-Selektor: alle Elemente mit dieser Klasse
`.site-nav a`	Nachfahre: Links innerhalb der Navigation

Schreibweise	Bedeutung
`a:hover`	Pseudo-Klasse: Link unter dem Mauszeiger
`--color-primary: #1d4ed8;`	Custom Property definieren (in `:root`)
`color: var(--color-primary);`	Custom Property verwenden
`padding` / `border` / `margin`	Innenabstand / Rahmen / Außenabstand
`display: flex;`	Kinder in einer Reihe anordnen
`justify-content` / `align-items`	Verteilung entlang der Haupt- / Querachse
`display: grid;`	Kinder in einem Raster anordnen
`grid-template-columns: repeat(2, 1fr);`	Zwei gleich breite Spalten
`gap`	Abstand zwischen Flex- oder Grid-Kindern
`@media (min-width: 40rem) { … }`	Regeln ab einer Mindestbreite

Viel Erfolg!